

BitFS: 基于区块链的点对点加密文件系统

摘要。 本文提出一种去中心化文件系统，将 Unix 文件系统语义映射到基于区块链的有向无环图上。所有文件默认使用椭圆曲线 Diffie-Hellman 方案 (Method 42) 加密存储，访问控制不取决于数据是否加密，而取决于谁能派生出解密密钥。层次确定性钱包镜像文件系统树结构，为元数据提供稳定的节点身份，同时确定性地派生每个版本的解密密钥。文件内容以密文的 SHA-256 哈希为键存储在内容寻址存储中，元数据则以 Metanet 交易的形式组织在 BSV 区块链上。无信任的数据交易通过哈希时间锁合约 (HTLC) 实现：卖方揭示密钥胶囊作为哈希原像以领取付款，买方使用揭示的胶囊派生解密密钥。系统以 SPV 模式运行，从不直接查询区块链。Agent 优先的 HTTP 接口使 AI Agent 能够通过内容协商和 x402 支付协议自主发现、浏览和购买内容。

1. 引言

现有的去中心化存储系统面临数据隐私与数据交易之间的根本矛盾。IPFS 等系统提供内容寻址存储，但不具备原生的加密或支付机制。Filecoin 增加了经济激励，但存储验证需要计算密集的零知识证明。中心化云存储使用方便，但需要信任单一实体来管理数据托管和访问控制。

BitFS 基于三个关键观察来解决这些矛盾：

1. **元数据与内容需要不同的保障。** 元数据（文件名、目录结构、访问策略）受益于区块链的不可变性和全局共识，而文件内容需要高效的存储和检索。两者应当分离。
2. **加密应当是默认的，而非可选的。** 与其在“加密”和“不加密”之间选择，不如所有数据都加密存储。公开数据与私有数据的区别仅在于密钥分发：公开数据使用可平凡派生的密钥，私有数据使用仅所有者知晓的密钥。
3. **AI Agent 是一等公民。** 随着 AI Agent 越来越多地自主与网络服务交互，文件系统应提供机器可读的接口，使 Agent 无需人类中介即可发现、浏览和交易。

BitFS 实现了一个 Unix 风格的文件系统：inode 是区块链公钥，目录项是 Metanet 交易载荷，所有文件内容使用确定性 ECDH 密钥派生加密。最终，一个单一的密码学框架 —— Method 42 [1] —— 同时提供了文件加密、访问控制、身份认证、无信任交易和存储证明。

2. Metanet 即文件系统

Metanet 协议 [2] 在 Bitcoin SV 区块链上定义了一个有向无环图 (DAG)，其中每个节点是一笔包含 `OP_RETURN` 输出的交易，携带节点公钥 `P_node` 和父交易的引用。边通过密码学建立：子交易必须包含由父节点私钥 `D_parent` 签名的输入，由比特币网络验证。

我们将 Unix 文件系统概念映射到此结构上：

Unix	BitFS
inode	<code>P_node</code> （压缩公钥，33 字节）
目录项	<code>ChildEntry(index, name, type, pubkey)</code>
inode 编号	<code>index</code> （每目录单调递增）
<code>..</code> （父目录）	载荷中的 <code>parent</code> 字段（父节点的 <code>P_node</code> ）

文件名仅存在于父目录的子节点列表中，节点自身不包含名称——这与 Unix 中 inode 不包含文件名完全一致。定义三种节点类型：

- **FILE** —— 持有 `encrypted_hash`、`content_hash`、MIME 类型和大小
- **DIR** —— 持有 `children` 列表和 `next_child_index` 计数器
- **LINK** —— 持有 `link_target`，类型为 **HARD**（TxID）、**SOFT**（`P_node`）或 **SOFT_REMOTE**（domain/path）

硬链接指向特定交易（固定版本）。软链接指向 `P_node`，始终解析为最新版本。远程软链接通过 DNS 解析跨越信任边界。

版本控制是内在的：同一 `P_node` 可出现在多笔交易中。区块高度最大（或同一区块内拓扑排序最后）的交易即为当前版本。所有历史版本保持可访问。

3. HD 密钥派生

每个节点的身份密钥 `P_node` 从 BIP32 [3] 层次确定性 (HD) 钱包派生，结构镜像文件系统层次：

```

m/44'/236'/0'          费用密钥链（所有 Vault 共享）
m/44'/236'/1'/0/0      Vault #0 根目录
m/44'/236'/1'/0/0/1    根目录的第一个子节点
m/44'/236'/1'/0/0/1/3  第一个子节点的第三个子节点
m/44'/236'/2'/0/0      Vault #1 根目录（独立树）

```

Vault 是一棵以 BIP32 账户为根的独立 Metanet 树。多个 Vault 共享同一种子，但维护完全独立的目录层次，每个 Vault 可绑定独立的域名。

此设计提供两个属性：（a）稳定的节点身份——目录的 `P_node` 在内容更新时不变，支持持久引用；（b）确定性恢复——整个密钥层次可从 BIP39 助记词重新生成，但交易数据需从备份恢复。

4. 统一加密

所有文件在存储前加密。每个文件版本的加密密钥使用 Method 42 [1]（基于 ECDH 的方案）派生：

1. $Df(0) = H[Da(0) \mid content_hash \mid file_index]$ (确定性派生)
2. $Pf(0) = Df(0) \times G$ (一次性公钥)
3. $point = Da(0) \times Pf(0) = Df(0) \times Pa(0)$ (ECDH 共享点)
4. $symmetric_key = SHA256(point.x)$ (AES-256-GCM 密钥)
5. $ciphertext = AES-GCM(plaintext, symmetric_key)$
6. $encrypted_hash = SHA256(ciphertext)$ (内容寻址标识)

其中 $Da(0)$ 是所有者的主私钥， $Pa(0) = Da(0) \times G$ 是对应公钥， $content_hash = SHA256(plaintext)$ 是原始内容的哈希， $file_index$ 是节点在 HD 树中的位置。

一次性密钥 $Pf(0)$ 从不存储。它可从 $Da(0)$ 和记录在 Metanet 载荷中的 $content_hash$ 确定性恢复。两个哈希都需要记录： $content_hash$ （明文哈希，用于密钥派生）和 $encrypted_hash$ （密文哈希，用于内容寻址）。两者不能合并：加密前不知道 $encrypted_hash$ ，而 $content_hash$ 正是派生加密密钥所需。

三种访问级别从同一机制中自然产生：

访问类型	密钥派生	谁能解密
私有	$Df(0) = H[Da(0) \mid content_hash \mid file_index]$	仅所有者
免费	$Df(0) = 1$ ，故 $key = SHA256(Pa(0).x)$	知道 $Pa(0)$ 的任何人
付费	标准 Method 42	买方，在 HTLC 交换后

对于免费数据，“平凡密钥技巧”将 $Df(0)$ 设为 1，使 $Pf(0) = G$ （生成元点）。对称密钥变为 $SHA256(Pa(0).x)$ 。由于 $Pa(0)$ 通过 DNS 公开，任何人都能计算解密密钥——但数据在磁盘上仍然是加密的，保持了统一的存储模型。

对于私有数据，整个 Metanet 载荷都用所有者的对称密钥加密，使文件名、大小、时间戳和目录结构在链上不可见。

5. 内容寻址存储

加密后的文件内容存储在本地内容寻址存储中，以 $encrypted_hash$ 为索引。这是一个扁平的键值映射：

```
~/.bitfs/data/{encrypted_hash} → 密文字节
```

Daemon 通过 HTTP 在 `GET /data/{hash}` 提供此内容。无需外部依赖（如 IPFS）。由于所有内容都已加密，存储层仅处理不透明的字节序列。去重自然发生：密文相同的两个文件共享同一 `encrypted_hash` 和单一存储副本。

6. 交易结构

每个文件系统操作产生一笔 Metanet 交易：

```
输入：
[0] 花费锁定到 P_parent 的 UTXO → Sig(D_parent) 创建 Metanet 边
[1] 花费费用密钥链 UTXO          → 支付矿工费

输出：
[0] OP_RETURN: <MetaFlag> <P_node> <TxID_parent> <Protobuf 载荷>
[1] P2PKH → P_node      (546 sat, 节点可花费输出)
[2] P2PKH → P_parent    (546 sat, 刷新父节点 UTXO)
[3] P2PKH → 找零地址
```

输出 2 至关重要：它刷新父节点的 UTXO，形成自持续 UTXO 链。每笔交易为父节点创建新的可花费输出，使下一次子节点操作无需预先充值节点地址。初始资金仅来自费用密钥链。

Protobuf 载荷编码节点类型、操作（CREATE/UPDATE/DELETE）、内容元数据、访问控制、目录子节点，以及关键词、描述和域名绑定等可选字段。

定价使用 `price_per_kb` 字段（satoshis/KB），支持目录继承：未设置价格的文件继承最近祖先的价格。总价由客户端计算：`ceil(price_per_kb × file_size / 1024)`。

7. 无信任数据交易

付费内容通过哈希时间锁合约 (HTLC) 原子交换购买。卖方完全无状态——不维护买家数据库、不保持会话状态、不记录购买历史。

1. 买方连接卖方 Daemon
2. Method 42 ECDH 握手 (双向身份验证)
3. 买方请求目标文件的 capsule_hash
4. 卖方计算:


```
capsule = ECDH_derived_key_material(Da(0), P_buyer)
capsule_hash = SHA256(capsule)
```
5. 卖方返回 capsule_hash + payment_address
6. 买方创建 HTLC 交易:


```
IF SHA256(preimage) == capsule_hash AND Sig(seller) → 卖方领取
ELSE IF timeout_expired AND Sig(buyer) → 买方退款
```
7. 买方广播 HTLC
8. 卖方观察到 HTLC → 揭示 capsule (原像) → 领取付款
9. 买方从链上读取 capsule → 派生解密密钥 → 解密文件

原子交换保证：卖方收到付款当且仅当买方收到密钥胶囊。无需可信中介。买方本地缓存密钥；后续访问仅从 Daemon 获取加密内容并在本地解密。

交换前的 Method 42 握手提供双向身份验证：

```

买方 → 卖方: P_buyer, nonce_b, timestamp
卖方 → 买方: P_seller, nonce_s, timestamp

双方计算: session_key = SHA256( ECDH(D_self, P_other).x || nonce_b || nonce_s )
双方验证: HMAC(session_key, "verify")

```

卖方的 `P_seller` 必须与通过 DNS 发布的 `Pa(0)` 一致，防止中间人攻击。只有 `D_seller` 的持有者才能计算出正确的会话密钥。

8. SPV 模式

BitFS 完全以简化支付验证 (SPV) 模式 [4] 运行。所有者将自己创建的所有交易及其 Merkle 证明存储在本地。访问者从所有者的 Daemon 获取元数据，从不查询区块链。

```

~/.bitfs/
├── wallet.db           加密的 HD 密钥 + UTXO 集合
├── txstore/{txid}.tx   完整交易数据
├── txstore/{txid}.proof Merkle 证明 (区块头 + 路径)
├── headers/           区块头链
└── cache/             访问者元数据 + 解密内容 + 密钥胶囊缓存

```

访问者通过检查 Merkle 证明与区块头链来验证收到的元数据。这消除了正常运行对区块链索引服务的依赖。第三方索引仅在所有者 Daemon 不可达时作为降级备用。

从种子恢复可还原 HD 密钥层次。交易数据需从备份恢复 —— 不执行区块链扫描。

9. DNS 解析与发布

任何 Metanet 节点（目录或文件）都可通过两种 DNS 记录绑定到域名：

<code>_bitfs_pubkey.example.com</code>	TXT	<code>"02a1b2c3..."</code>	(P_node 身份)
<code>_bitfs._tcp.example.com</code>	SRV	<code>10 60 443 cdn1.example.com</code>	(服务端点)
<code>_bitfs._tcp.example.com</code>	SRV	<code>20 100 443 backup.example.com</code>	

双向验证确保一致性：DNS TXT 记录指向 `P_node`，Metanet 载荷的 `domain` 字段记录绑定的域名。两者必须一致。

SRV 记录通过 `priority` 和 `weight` 字段提供原生负载均衡，实现类 CDN 的分发而无需应用层路由。可声明多个端点，客户端连接优先级最高（`priority` 数值最小）的可用服务器。

URI 解析流程如下：

```
bitfs://example.com/docs/readme.txt
→ DNS TXT → P_node
→ DNS SRV → 端点列表（按 priority/weight 排序）
→ 连接最优端点，Method 42 握手
→ 请求 Metanet 元数据（SPV）
→ 验证双向域名绑定
→ 遍历目录树 → 返回内容
```

通过公钥直接寻址（`bitfs://<pubkey>/path`）可完全绕过 DNS。

10. Agent 优先接口

Daemon 实现内容协商，从相同端点同时服务人类和 AI Agent：

Accept 头	响应
<code>text/html</code>	包含 WebMCP 工具声明的 HTML 页面
<code>text/markdown</code>	自描述的 CLI 使用指南
<code>application/json</code>	结构化元数据

对于付费内容，HTTP 402 响应包含支付元数据头（`X-Price`、`X-Price-Per-KB`、`X-File-Size`、`X-Invoice-Id`）和格式适配的正文。人类看到付费墙。浏览器中的 AI Agent 发现 WebMCP 工具声明并自主调用支付。CLI Agent 收到命令模板（`bget --buy bitfs://...`）并执行。

核心洞察：拥有钱包的 Agent 可以透明地完成微支付。402 状态码从传统的访问壁垒变为可编程的支付 API。

11. 双模式命令行

系统提供两类工具，体现 Unix 哲学：

只读工具（`b*`）：无状态，无需钱包，服务访问者。

- `bls` —— 列目录（类似 `ls`）
- `bcat` —— 输出文件内容到 `stdout`（类似 `cat`）
- `bget` —— 下载文件到本地（类似 `wget`）
- `bstat` —— 显示节点元数据（类似 `stat`）
- `btree` —— 递归目录树（类似 `tree`）

读写命令（`bitfs`）：需要钱包和私钥，服务所有者。

- `bitfs put/mkdir/rm/mv/cp/link` —— 文件系统操作
- `bitfs sell/encrypt/decrypt` —— 交易与访问控制
- `bitfs vault/wallet/publish/daemon` —— 基础设施管理
- `bitfs shell` —— FTP 风格的链上文件系统交互 REPL

所有工具支持 `--json` 输出以供程序消费，以及 `--offline` 模式使用缓存数据。

12. 激励层

作为未来扩展，提供文件加密的 Method 42 ECDH 机制同样可实现高效的存储证明。向存储提供者分发内容时，每份副本使用提供者特定的 ECDH 密钥重新加密：

```
ECDH(Da(0), P_providerA) → key_A → AES(ciphertext, key_A) → hash_A  
ECDH(Da(0), P_providerB) → key_B → AES(ciphertext, key_B) → hash_B
```

每个提供者存储的数据在密码学上是唯一的。验证仅需简单的 Merkle 挑战-响应：请求随机块，对照链上记录的 Merkle 根验证哈希。这替代了 Filecoin 等系统使用的计算密集型零知识证明，将验证从数小时的 GPU 计算降低到毫秒级的 ECDH + AES 操作。

BitFS 子链拥有自己的工作量证明共识、代币经济（2100 万固定供应、减半计划）和通过 `OP_RETURN` Merkle 根定期锚定到 BSV 主链，为存储提供者激励提供经济基础。

13. 结论

我们提出了一种去中心化文件系统，将 Unix 语义映射到区块链支持的元数据上，默认加密所有内容，并通过原子交换实现无信任的数据交易。系统设计围绕单一密码学原语 —— ECDH 密钥派生 —— 统一构建，它同时提供加密、认证、支付验证和存储证明。通过将

AI Agent 视为一等用户并以 SPV 模式运行而不查询区块链，BitFS 为自主 Agent 以机器速度与去中心化服务交互的未来而设计。

参考文献

- [1] C. S. Wright, "An Immutable File and Data Store," nChain, 2025. Method 42: 使用 ECDSA/secp256k1 的确定性密钥派生实现逐文件加密。
- [2] nChain, "The Metanet Technical Summary v1.0," 2020. 基于区块链的有向无环图，用于组织数据关系。
- [3] P. Wuille, "BIP32: Hierarchical Deterministic Wallets," Bitcoin Improvement Proposals, 2012.
- [4] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. 第 8 节：简化支付验证。
- [5] GB2608179A, "Multi-level Blockchain," UK Intellectual Property Office, 2025. 通过载体对和多层交易在核心区块链上嵌入二级数据链。